Maleeka Raddygala
maleeka@stanford.edu
Token Compression Challenge, The Token Company
GitHub: github.com/NullPointerNinja007/EvidenceCodec—Context-Compression-Algorithm

EvidenceCodec: Query-Conditioned Evidence Selection for Token Compression

June 21, 2026

Abstract

EvidenceCodec is a query-conditioned token compression system designed to reduce long input contexts while preserving answer-critical evidence. The core design choice is to avoid abstractive summarization and instead select raw text chunks that can be passed directly to a downstream model. The implemented pipeline combines deterministic structure-preserving chunking, high-recall hybrid candidate generation, ModernBERT-large cross-encoder reranking, and budgeted evidence selection with a deterministic RiskGuard safety layer. In a few-hour challenge setting, full HotpotQA training artifacts were built with parallel Modal CPU workers, and a time-boxed ModernBERT-large run produced six checkpoints. On a 1,000-example HotpotQA validation slice, EvidenceRecall@Budget improved from 0.701 at step 50 to 0.872 at step 300, while AllGoldSelected@Budget improved from 0.484 to 0.759. These results show that the architecture reached a strong initial evidence-selection baseline despite limited training time.

Introduction

Long-context systems often waste tokens on material that is not needed for a specific query. A token compression method should therefore do more than shorten text; it should preserve the information required to answer the question. For evidence-heavy tasks, the critical failure mode is silent evidence loss. A compressed context may look fluent and relevant while omitting a date, negation, exception, quantity, named entity relation, or other small fact that changes the answer.

EvidenceCodec addresses this by treating compression as evidence selection. Given a query q and a long context D, the system chooses a subset of raw text chunks $S \subset D$ under a token budget B. The compressed output is not a summary. It is a stitched set of original text spans selected because they are likely to contain answer-supporting evidence.

This report describes the implemented architecture, the training and evaluation run, and the final metrics. The work was completed under a tight challenge deadline of only a few hours. Because of that constraint, the system was engineered to reach a working end-to-end result quickly: full artifacts were built, the reranker was trained for 300 optimizer steps, six checkpoints were saved, every checkpoint was evaluated, and metric plots were generated. The run did not have enough time for a full epoch, QASPER adaptation, or LongBench evaluation.

System Objective

The target behavior is query-conditioned compression:

$$f(q, D, B) \to C,$$

where q is the user question, D is the original long context, B is the output token budget, and C is the selected compressed context. The system should maximize evidence retention under budget, not semantic smoothness.

The practical objectives were:

1. keep the output auditable by preserving raw text;
2. maximize recall of gold supporting evidence in the candidate pool;
3. train a reranker that ranks true evidence above distractors;
4. select high-utility evidence under a strict token budget;
5. expose metrics that show how quality changes with training step.

Architecture Overview

Figure 1 gives the high-level system architecture. The pipeline is deliberately modular: each stage has a narrow responsibility and produces inspectable artifacts.

Implementation Interfaces

The code is organized around three small data objects. A `Chunk` stores `chunk_id`, raw `text`, `token_count`, optional character offsets, breadcrumb headings, and metadata. A `Candidate` wraps a chunk with `retrieval_score`, normalized `risk_score`, optional `rerank_score`, a source string, and a score dictionary. Its utility is the reranker score when available and the retrieval score otherwise. A `CompressionResult` stores the query, final compressed

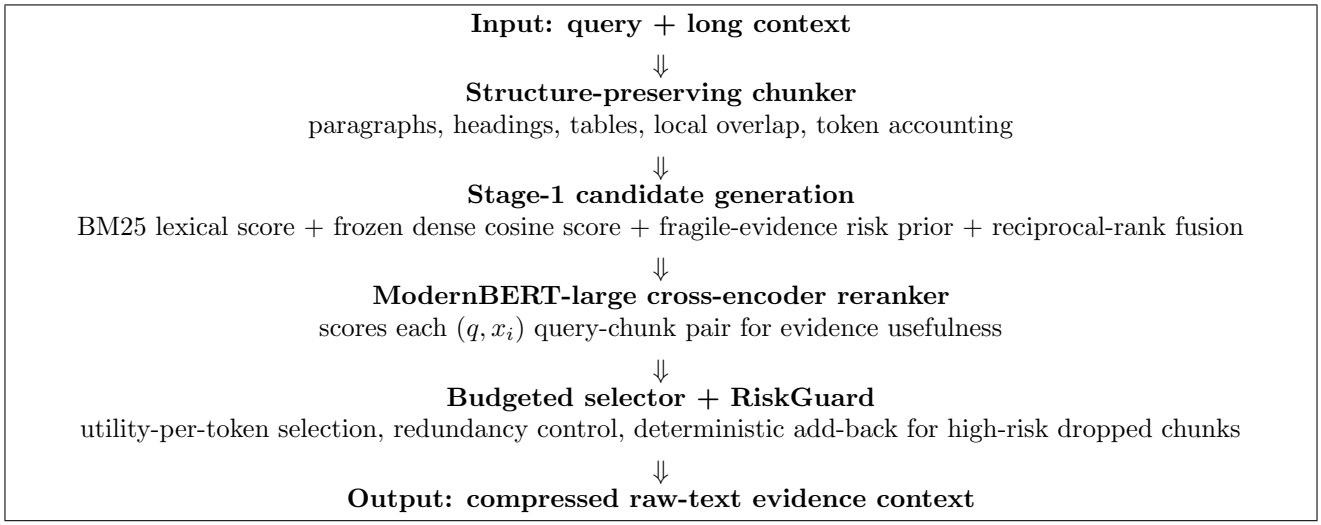

Figure 1: EvidenceCodec architecture. The system compresses by selecting raw evidence spans rather than generating summaries.

raw-text context, selected chunks, dropped chunks, and audit metrics.

This interface is important because each stage can be inspected independently. The chunker does not know about ModernBERT. Stage 1 does not know about optimizer state. The selector does not need to know whether utility came from the cross-encoder, the hybrid retrieval score, or a future cheap student. That makes it possible to run retrieval-only smoke tests, B200 quality mode, and offline evaluation with the same downstream selector.

Deterministic Chunking

The implemented chunker is hierarchical and conservative. It first converts input records into `TextUnit` objects. Each unit has text, a stable unit id, breadcrumb headings, metadata, and a `group_key`. The packer then greedily groups adjacent units into chunks. It flushes the buffer when the structural group changes, when the hard token limit would be exceeded, or when the soft limit would be exceeded at a safe boundary.

The default constants are:

$$L_{\text{target}} = 160, \quad L_{\text{soft}} = 192, \quad L_{\text{hard}} = 224, \quad o = 32.$$

If an individual unit is too large, it is split by sentence boundaries. If sentence splitting cannot keep the text under the hard limit, token windows are used as a fallback. Breadcrumbs are prepended as a heading prefix such as `title > section`; this gives the reranker local section context without requiring a larger chunk.

Boundary safety is rule-based. The chunker avoids flushing across a local boundary when the combined boundary window contains markers such as `not`, `unless`, `except`, `only if`, `provided that`, `cannot`, `before`, `after`, `must`, or `required`. The goal is not semantic reasoning; the goal is to avoid separating a claim from the qualifier that changes its meaning.

The dataset adapters make the chunker exact enough for supervision:

- **HotpotQA:** every source sentence becomes a unit with breadcrumb equal to the article title. Metadata stores `title`, `title_index`, `sent_id`, and a `hotpot_ref` key of the form `title::sent_id`. A chunk is positive if any packed source unit's `hotpot_ref` appears in the supporting-fact set.
- **QASPER:** the document is converted into abstract units, section-paragraph units, and figure/table-caption units. A chunk is positive when normalized highlighted evidence or evidence paragraphs appear in the chunk, or when a long evidence span has at least 0.72 token overlap.
- **LongBench:** records are paragraph chunked from the `context` field and kept evaluation-only because most tasks do not expose chunk-level evidence labels.

Stage-1 Candidate Generation

Stage 1 is designed for recall. It deliberately uses rank fusion instead of treating BM25, dense similarity, and risk scores as if they were calibrated to the same scale.

BM25 tokenization uses a retrieval pattern designed to preserve useful code-like and number-like tokens:

```
[A-Za-z0-9_.$%/-]+
```

Dense scoring has two interchangeable backends. The fast deterministic backend is a normalized hashing vectorizer with 4,096 features and character word-boundary n-grams of length 3 to 5. The semantic backend is a frozen sentence-transformer scorer using `BAAI/bge-small-en-v1.5`. Both return cosine-like scores.

For every query and chunk list, the implementation computes raw BM25, raw dense, and raw risk scores, then min-max normalizes them within the query. A diagnostic hybrid score is stored:

$$h_i = 0.45\hat{b}_i + 0.45\hat{d}_i + 0.10\hat{r}_i.$$

Module	Main responsibility	Exact implementation detail
`units.py`	Pack structure units into chunks	Greedy contiguous packing with target 160 tokens, soft max 192, hard max 224, and 32-token overlap.
`dataset_chunkers.py`	Dataset-specific units and labels	HotpotQA sentence units with title breadcrumbs; QASPER abstract, section paragraphs, and figure/table captions; LongBench paragraph units for evaluation.
`stage1.py`	High-recall candidate pool	BM25 top 96, dense top 96, risk top 32, RRF cap 256 with $c = 60$ and risk weight 0.75.
`cross_encoder.py`	ModernBERT-large scorer	Query max 96 tokens, pair length 512, bf16, final-layer CLS state, dropout 0.1, scalar linear head.
`budget.py`	Budgeted selection	Greedy utility per token with $\alpha = 0.9$ and Jaccard redundancy penalty 0.15.
`risk_addback.py`	Deterministic rescue	Eligible dropped chunks require utility >= 0.35, risk >= 0.55, and fit in a 10 percent add-back reserve.
`checkpoint_run.py`	Time-boxed full run	Resume-aware B200 training, six checkpoint schedule, parallel B200 checkpoint evaluation, CSV/JSONL/plot outputs.

Table 1: Concrete architecture-to-code mapping. The Modal wrappers are intentionally thin and import these implementation modules rather than containing the main logic.

The actual candidate set is RRF fused from three top lists:

$$C = \mathrm{RRF}(\mathrm{Top}_{96}^{BM25}, \mathrm{Top}_{96}^{dense}, \mathrm{Top}_{32}^{risk}),$$

with

$$\mathrm{RRF}(i) = \sum_m \frac{\alpha_m}{60 + \mathrm{rank}_m(i)},$$
$$\alpha_{BM25} = 1.0, \quad \alpha_{dense} = 1.0, \quad \alpha_{risk} = 0.75.$$

The fused list is capped at 256 candidates. The candidate JSONL stores the ids, rank, source memberships, RRF score, raw component scores, normalized risk, hybrid score, token count, candidate recall, and whether all gold chunks were covered.

Fragile-Evidence Risk Prior

The risk prior is a cheap guard against dropping small facts that carry high semantic load. It is query-aware only through entity, heading, and numeric overlap; it is not a second neural model. The implemented risk score is:

$$\begin{aligned} r_i = {} & 2.0 f_{neg} + 2.0 f_{exc} + 1.5 f_{num} + 1.5 f_{date} + 1.5 f_{money} \\ & + 1.2 f_{cmp} + 1.5 f_{code} + 1.5 f_{table} + 1.2 f_{eq} \\ & + 1.0 f_{entity} + 1.0 f_{qnum} + 0.5 f_{heading}. \end{aligned}$$

The raw score is then min-max normalized within the query before being attached to candidates.

Feature	Trigger
`negation`	no, not, never, cannot, without
`exception`	unless, except, only if, provided that
`number/date`	numbers, percentages, years, month names
`money`	currency symbols or USD/EUR/GBP values
`comparison`	greater, less, before, after, minimum, first, later
`code`	inline code, function calls, file paths
`table/equation`	Markdown table rows, table metadata, equation symbols
`overlap`	query entity, number, or heading overlap

Table 2: Risk prior features used in Stage 1 and RiskGuard.

ModernBERT-Large Cross-Encoder

The quality scorer is a cross-encoder, not a bi-encoder. For each candidate chunk x_i, the tokenizer first truncates the query to at most 96 tokens. The remaining budget is assigned to the chunk so that the final pair fits inside 512 tokens. The packed input is:

`[CLS] query [SEP] chunk [SEP].`

The model uses `answerdotai/ModernBERT-large` through Hugging Face `AutoModel` with `trust_remote_code=True`. In bf16 mode, the encoder is loaded with bfloat16 weights. The classifier is intentionally simple: final hidden state at position 0, dropout 0.1, then a one-dimensional linear layer:

$$h_i = \mathrm{Enc}_\theta(q, x_i)_0, \qquad s_i = w^\top \mathrm{Dropout}(h_i) + b.$$

At inference time, the logit is converted to $p_i = \sigma(s_i)$ and stored as `rerank_prob`. The selected utility becomes the rerank probability, while the raw

logit remains available for analysis. Checkpoints save the encoder, tokenizer, `classifier.pt`, and an `evidence_codec_reranker_config.json` file, so evaluation can reload exactly the trained head and configuration.

Training Data and Objective

Training pairs are constructed from the chunk and candidate JSONL files. For each example, positives are candidate ids whose projected chunk label is 1. Negatives are the top non-gold Stage-1 candidates, keeping up to $\max(8, 8 \cdot |P|)$ negatives. Each example id receives a stable integer group id. Training microbatches are query groups, and gradient accumulation windows combine groups until the window reaches the requested effective batch size of 128 pairs.

The loss is computed inside each optimizer step with microbatch weighting by pair count. This matters because query groups can have different numbers of positives and negatives. The objective is:

$$L = 1.0 L_{\text{BCE}} + 0.5 L_{\text{rank}},$$

where

$$L_{\text{rank}} = \frac{1}{|G|} \sum_{g \in G} \frac{1}{|P_g||N_g|} \sum_{p,n} \max(0, 0.2 - s_p + s_n).$$

Groups without both a positive and a negative contribute zero ranking loss but still contribute BCE. The optimizer is AdamW with learning rate $2 \cdot 10^{-5}$, weight decay 0.01, linear warmup ratio 0.06, bf16 autocast on CUDA, and max gradient norm 1.0.

The checkpoint trainer is resume-aware. If a previous checkpoint exists and overwrite is false, it reloads the latest checkpoint and trainer state. Checkpoint steps are generated by evenly dividing the total requested steps. For the completed 300-step time-boxed run with six checkpoints, the saved steps were 50, 100, 150, 200, 250, and 300.

Budgeted Selection and RiskGuard

The selector receives candidates after reranking. In default serving configuration the total budget is 4,096 tokens. In the checkpoint evaluation reported here, the budget was 512 tokens to stress compression. The selector reserves 10 percent of the budget for risk add-back. Therefore, with a 512-token evaluation budget, the main selector sees 461 tokens and RiskGuard may spend up to 51 tokens.

The main selector greedily recomputes a priority after each selected chunk:

$$\text{priority}_i = \frac{\max(0, u_i - \eta \max_{j \in S} \text{Jaccard}(x_i, x_j))}{\text{tokens}(x_i)^{0.9}},$$

with $\eta = 0.15$. This is a practical approximation to utility maximization under a hard token budget. A chunk that would exceed the remaining budget is skipped. The Jaccard redundancy term uses lowercase alphanumeric content tokens and prevents near-duplicate chunks from consuming the budget.

RiskGuard then examines the dropped candidates. A dropped chunk is eligible when:

$$u_i \geq 0.35, \qquad \hat{r}_i \geq 0.55.$$

Eligible chunks are sorted by normalized risk per token and added back only if they fit both the total budget and the add-back reserve. The final selected chunks are sorted back into document order before prompt assembly. This preserves readability and avoids presenting evidence in model-score order.

Engineering and Artifact Pipeline

The implementation was designed around Modal volumes and parallel jobs. Large datasets, processed artifacts, checkpoints, and evaluation outputs were not stored as local training data. They were stored in isolated Modal volumes:

- `/data` for raw Arrow downloads, chunk JSONL, candidate JSONL, and shard outputs;
- `/models` for ModernBERT checkpoints and trainer state;
- `/runs` for checkpoint metrics, CSVs, JSONL logs, and plots;
- `/cache` for Hugging Face datasets, model weights, and transformer caches.

The Modal volume names were isolated under the `evidencecodec-ccalg-20260621` prefix so the run would not overwrite unrelated account volumes. Every writing function calls `commit_all()`, and functions that read outputs from other containers call `reload_all()` first. This matters for correctness when preprocessing, training, and evaluation run in separate containers.

The full HotpotQA artifact builder uses 100 CPU containers. Each shard maps a contiguous row range from the raw Arrow files, chunks the example, projects labels, immediately builds the Stage-1 candidate record, writes paired shard JSONL files, and leaves a done marker. The merge step refuses to run until all shard done markers exist. This design avoids writing a huge chunk file and then launching a separate candidate-generation pass over it.

The GPU path used only B200 instances. One B200 function trained ModernBERT-large with checkpoints. Evaluation used B200 workers with `max_containers=8`; each worker reloaded a checkpoint, reranked the validation candidates, ran the selector at a 512-token budget, and returned metrics. A final Modal function wrote `metrics_by_checkpoint.jsonl`, `metrics_by_checkpoint.csv`, and one PNG per metric.

End-to-End Algorithms

This section states the exact online and offline algorithms implemented by the system. The deployed compressor does

not call a teacher LLM, does not send hidden states to the final LLM, and does not perform abstractive rewriting. All neural computation is internal scoring over raw-text candidates.

Quality-Mode Compression

Given context D, query q, and token budget B, quality-mode compression runs:

1. Build deterministic chunks $X = \{x_1, \ldots, x_n\}$ with stable ids, token counts, breadcrumbs, and provenance metadata.
2. Score every chunk with BM25, dense cosine-like similarity, and fragile-evidence risk.
3. Form three top lists: lexical top 96, dense top 96, and risk top 32.
4. Fuse the lists with weighted RRF and keep at most 256 candidates.
5. Pack each (q, x_i) pair for ModernBERT-large using a 96-token query cap and 512-token pair cap.
6. Score candidates with the cross-encoder and convert logits to sigmoid probabilities.
7. Run budgeted selection with a 10 percent RiskGuard reserve.
8. Sort selected chunks back into original document order and join them with blank lines.

The output is therefore:

$$
\begin{aligned}
X &= \mathrm{Chunk}(D), \\
K &= \mathrm{Stage1}(X, q), \\
R &= \mathrm{Rerank}(K, q), \\
C &= \mathrm{JoinByDocumentOrder}(\mathrm{Select}_B(R)).
\end{aligned}
$$

The final prompt assembly is simple text assembly: `Context:` followed by selected raw chunks, then `Question:` followed by the query. This keeps the compressor model-agnostic: any downstream chat or completion model can consume the compressed context.

Fallback and Smoke-Test Modes

The same selector supports lower-cost modes. If `use_reranker=False`, the pipeline skips ModernBERT and uses Stage-1 scores as candidate utility. If explicit candidate scores are supplied, `compress_with_scores` creates candidates directly from chunk ids and provided scores. These paths were useful for tests because they verify chunking, risk scoring, and budget behavior without GPU dependencies.

The architecture also leaves room for the distilled student described in the project docs. The student was not necessary for the final time-boxed run, because the challenge objective was to prove the quality path first. The codebase still has a clean place for it: it can learn from saved cross-encoder scores and output the same candidate utility field consumed by the selector.

Training Artifact Construction

The training artifact algorithm is intentionally one-pass per shard:

1. Read a contiguous slice of HotpotQA Arrow rows from `/data/raw/hotpotqa/train`.
2. Convert each row into HotpotQA sentence units with article-title breadcrumbs.
3. Pack units into chunks with the shared chunking defaults.
4. Project supporting facts onto chunks using `title::sent_id` metadata.
5. Run Stage 1 immediately for that record, producing a candidate record aligned with the chunk record.
6. Write `chunks.part-xxxxx.jsonl`, `candidates.part-xxxxx.jsonl`, and a shard done marker.

This is faster and safer than building all chunks first and then launching a second full pass. It also guarantees that every candidate record was built from the same chunk record emitted by that shard. The merge step concatenates shards in shard id order, so train and candidate JSONL files remain line-aligned.

Checkpoint Evaluation Algorithm

Each checkpoint evaluation repeats the actual compression path, not only the ranking metric path. For each validation example, the evaluator loads the saved checkpoint, reranks Stage-1 candidates, computes MRR and nDCG from the reranked order, runs the 512-token selector, and then computes evidence retention from selected ids. This matters because a checkpoint can have slightly better MRR but worse budgeted evidence recall. That is exactly what happened: step 250 had the best reranking metrics, while step 300 had the best compressed-context evidence metrics.

Artifact Contracts

The architecture is built around line-aligned JSONL contracts. These contracts made it possible to parallelize preprocessing, resume training, and evaluate checkpoints without hidden local state.

Chunk Record Contract

A chunk record represents one query/document example. It contains:

- `example_id`, `dataset`, `split`, and optional subset metadata;
- `query` and answer strings when the dataset provides them;
- `chunks`, each with id, text, token count, breadcrumbs, and source-unit metadata;
- `labels`, aligned one-to-one with chunks when supervision exists;
- `gold_chunk_ids`, derived from labels;

- `gold_evidence`, preserving original evidence references or spans;
- summary metadata such as positive chunk count and total chunk count.

For HotpotQA, label projection is exact at sentence level because the dataset exposes supporting fact titles and sentence ids. For QASPER, projection is necessarily overlap-based because evidence may be paragraph-level or highlighted text. That distinction is explicit in the data layer instead of hidden inside the model.

Candidate Record Contract

A candidate record is line-aligned with its chunk record. It stores `candidate_ids`, detailed candidate score rows, ranked lists, and candidate recall diagnostics. Each candidate score row includes:

$$\{\text{rank}, \text{sources}, \text{RRF}, \text{BM25}, \text{dense}, \text{risk}, \hat{r}, h_i, tokens\}.$$

This makes Stage 1 auditable. If a gold chunk is missing, the failure is visible before training. If a risky chunk is present only because of the risk list, the `stage1_sources` field shows that directly.

Reranker Pair Contract

Training examples are created lazily from chunk and candidate records. A pair contains `example_id`, `chunk_id`, query, chunk text, binary label, Stage-1 rank, and group id. The group id is the unit used by the pairwise margin loss. This makes negatives same-query negatives by construction, rather than random corpus negatives only.

Checkpoint Contract

Each saved checkpoint directory contains the ModernBERT encoder files, tokenizer files, `classifier.pt`, `trainer_state.pt`, and the reranker config JSON. The trainer state stores optimizer, scheduler, current step, total steps, and reranker config. This is why the training function can resume instead of starting over when a Modal retry or timeout occurs.

Architecture Rationale

The main architectural choice is a multi-stage extractive compressor rather than a learned summarizer. That choice was made for three reasons. First, raw text keeps provenance visible: an evaluator can inspect exactly which evidence survived. Second, the final LLM receives normal text, so the compressor is not tied to one model provider or embedding interface. Third, supporting-fact datasets supervise evidence selection naturally; they do not directly supervise faithful abstractive compression.

The chunker is deterministic because learned chunking would add training complexity without solving the bottleneck. A chunker only needs to preserve local coherence and not destroy evidence boundaries. Stage 1 is hybrid because no single cheap retriever is reliable enough: BM25 catches lexical matches, dense scoring catches paraphrase-like matches, and the risk list catches fragile facts that may not look semantically central. The cross-encoder is where semantic judgment belongs because it jointly attends over the query and candidate text. The selector is token-aware because a pure relevance ranker can waste budget on long, redundant chunks.

This boundary also gives clear failure diagnosis. If gold evidence is absent from Stage 1, the issue is chunking, retrieval, or risk rescue. If gold evidence is present but ranked low, the issue is the reranker. If gold evidence is ranked high but missing after compression, the issue is selector budget, token accounting, or redundancy/risk thresholds. This separation was useful during the challenge because there was not enough time for broad end-to-end debugging.

Experimental Setup

Table 4 summarizes the completed run.

A full epoch would have required about 8,006 optimizer steps. The completed 300-step run therefore covered only about 3.7 percent of one epoch. This was an intentional time-boxed decision: the goal was to produce a complete trained baseline, checkpoint curve, and report within a few hours rather than to exhaust the dataset.

Evaluation Metrics

The report uses two classes of metrics.

Reranking metrics. MRR@10 and nDCG@10 measure whether gold evidence is ranked near the top. Recall@32, Recall@64, and Recall@128 measure whether gold evidence remains recoverable in the top candidate set.

Compression metrics. EvidenceRecall@Budget measures the fraction of gold evidence chunks retained after budgeted selection. AllGoldSelected@Budget measures the stricter event that every gold chunk for an example survives compression. Average selected tokens and selected chunk count describe the compression behavior.

The most important metric for this system is EvidenceRecall@Budget, because the system's output is the selected compressed context that will be given to a downstream model.

Results

Table 5 shows the checkpoint metrics. The model improved quickly from step 50 to step 150 and then plateaued.

From step 50 to step 300, MRR@10 improved from 0.760 to 0.928, a 22.2 percent relative gain. nDCG@10 improved from 0.769 to 0.895, a 16.4 percent relative gain. EvidenceRecall@Budget improved from 0.701 to 0.872,

Artifact path	Purpose
`/data/processed/chunks/hotpotqa/train.jsonl`	Full HotpotQA train chunk records, one line per example.
`/data/processed/candidates/hotpotqa/train.k256.jsonl`	Line-aligned Stage-1 train candidates capped at 256.
`/data/processed/chunks/hotpotqa/validation.sample1000.jsonl`	Validation chunk records for the time-boxed evaluation slice.
`/data/processed/candidates/hotpotqa/validation.sample1000.k256.jsonl`	Line-aligned validation candidate records.
`/models/rerankers/modernbert-large-hotpotqa-full/checkpoint-step-*`	Six B200-trained ModernBERT-large reranker checkpoints.
`/models/rerankers/modernbert-large-hotpotqa-full/training_summary.json`	Pair count, optimizer steps, checkpoint list, and train log path.
`/runs/evaluations/modernbert-large-hotpotqa-full/metrics_by_checkpoint.*`	Checkpoint metrics in JSONL and CSV form.
`/runs/evaluations/modernbert-large-hotpotqa-full/plots/*.png`	Metric curves used in this report.

Table 3: Final Modal artifact contract. These paths are on Modal volumes, not local training-data directories.

Item	Value
Training dataset	HotpotQA train
Train records	90,447
Train chunks/candidates	1,072,054
Training pairs	1,069,179
Validation slice	1,000 HotpotQA examples
Candidate cap	256
Model	answerdotai/ModernBERT-large
Precision	bf16
Sequence length	512
Effective batch size	128 pairs
Optimizer	AdamW
Learning rate	2e-5
Training budget	300 optimizer steps
Checkpoints	6 checkpoints

Table 4: Run configuration.

a 24.5 percent relative gain. AllGoldSelected@Budget improved from 0.484 to 0.759, a 56.8 percent relative gain.

The best reranking checkpoint was step 250, with MRR@10 of 0.930 and nDCG@10 of 0.895. The best compression checkpoint was step 300, with EvidenceRecall@Budget of 0.872 and AllGoldSelected@Budget of 0.759. Since the deployed compressor cares most about retained evidence under budget, step 300 is the recommended checkpoint.

Analysis

The results show three important patterns.

Candidate recall was solved on the validation slice. Recall@32, Recall@64, and Recall@128 were all 1.000. This is exactly what Stage 1 is supposed to accomplish. It means the gold evidence was usually present before reranking and selection, so failures were not caused by the retrieval stage dropping evidence too early.

The reranker learned rapidly. The biggest jump happened between steps 50 and 100. MRR@10 moved from 0.760 to 0.901, and EvidenceRecall@Budget moved from 0.701 to 0.834. This suggests that ModernBERT-large is well matched to the pairwise evidence selection task and can learn useful ranking behavior quickly.

The budgeted selector became more efficient. Average selected chunks decreased from 5.15 to 4.76 while EvidenceRecall@Budget increased. The system was not merely selecting more text. It was selecting better text. Average used tokens increased only modestly from 411 to 422, while all-gold coverage improved by 27.5 absolute percentage points.

Training losses support the same conclusion. Comparing the first 50 and last 50 optimizer steps of the completed 300-step run, total loss dropped by about 50.8 percent, BCE loss by about 49.5 percent, and ranking loss by about 57.8 percent. Both the classifier and the pairwise ranking objective were improving.

Limitations

The main limitation is training time. The project only had a few hours for implementation, artifact generation, training, evaluation, graphing, and reporting. The final run was capped at 300 optimizer steps so that it could produce at least five checkpoints, evaluate all checkpoints, and render plots before the deadline. A full epoch would be about 8,006 optimizer steps, so this run did not train long enough to claim convergence.

The evaluation was also limited to a 1,000-example

Step	MRR@10	nDCG@10	Recall@32	EvidenceRecall@Budget	AllGoldSelected@Budget	Used Tokens	Chunks
50	0.760	0.769	1.000	0.701	0.484	411.4	5.15
100	0.901	0.870	1.000	0.834	0.689	418.4	4.92
150	0.921	0.888	1.000	0.867	0.751	422.1	4.79
200	0.927	0.892	1.000	0.867	0.750	422.1	4.76
250	0.930	0.895	1.000	0.870	0.755	422.1	4.77
300	0.928	0.895	1.000	0.872	0.759	422.1	4.76

Table 5: Checkpoint metrics on 1,000 HotpotQA validation examples. Recall@64 and Recall@128 were also 1.000 at every checkpoint.

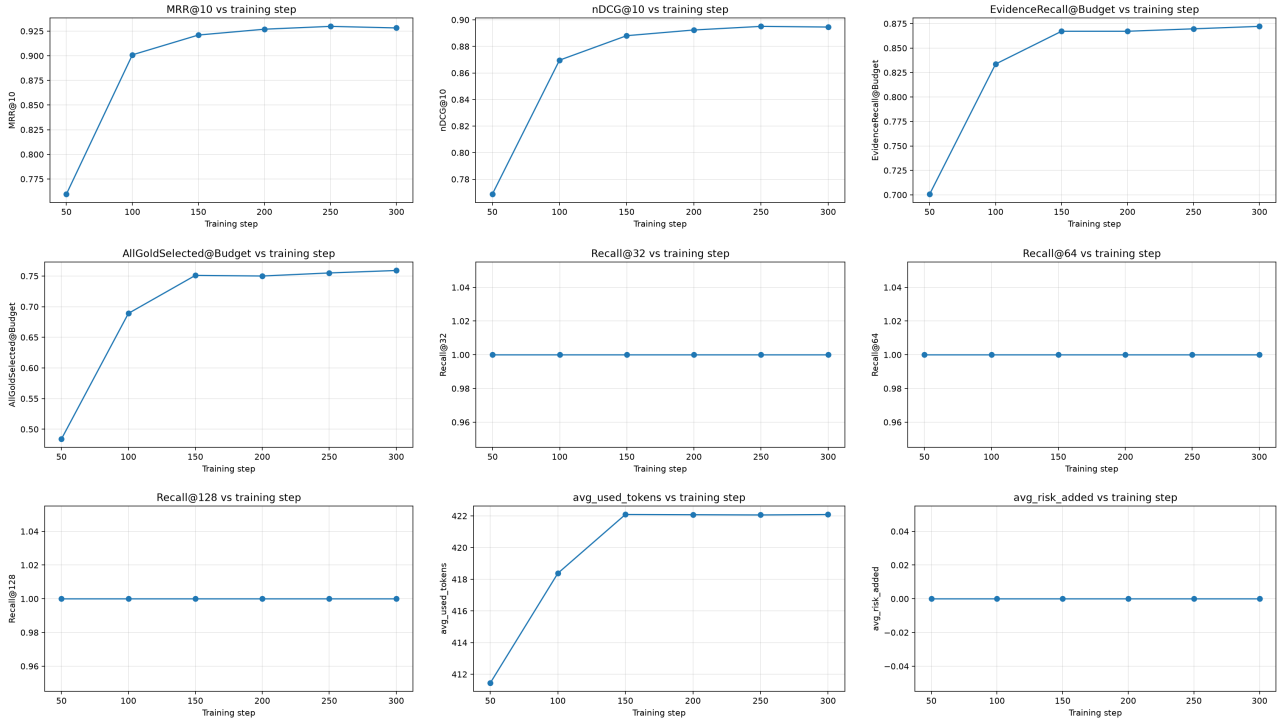

Figure 2: Metric curves across the six saved checkpoints.

HotpotQA validation slice. This is enough to show that the pipeline works and that training improved the model, but it is not a complete benchmark. Full HotpotQA validation, QASPER, and LongBench should be run before making broader claims.

Finally, RiskGuard did not activate on this validation slice. The implementation exists, but the reported slice did not stress the kinds of fragile facts RiskGuard is meant to protect. A dedicated stress set with numbers, dates, negations, legal/policy exceptions, and code identifiers is needed.

Conclusion

EvidenceCodec produced a working evidence-preserving token compression pipeline under a tight challenge deadline. The architecture successfully separated recall, semantic reranking, and budgeted selection. The completed run built full HotpotQA artifacts, trained ModernBERT-large on the full artifact set for a capped 300 steps, saved six checkpoints, evaluated all checkpoints, and generated metric plots.

The final checkpoint retained 87.2 percent of annotated evidence under budget and selected all gold evidence for 75.9 percent of validation examples. Given that this was only 3.7 percent of a full epoch, the result is a strong baseline rather than a final ceiling. With more time, the next step would be longer training followed by full validation and cross-domain evaluation.